

# RTL Design Sherpa

# APB PIC 8259 Micro-Architecture Specification 1.0

## January 4, 2026

# Table of Contents

1 Pic 8259 Mas Index..... 7

2 pic_8259 – Overview..... 8

2.1 Overview..... 8

2.1.1 Parameters..... 9

2.1.2 Applications..... 9

2.2 Functional Description..... 10

2.2.1 Figure 1.1: PIC 8259 Block Diagram..... 10

2.2.2 Register Summary..... 10

2.2.3 Interrupt Priority..... 11

2.2.4 Priority Modes..... 12

2.3 Waveforms..... 13

2.3.1 Waveform 1.1: Interrupt Request..... 13

2.3.2 Waveform 1.2: Interrupt Acknowledge Sequence..... 13

2.3.3 Waveform 1.3: End-of-Interrupt (EOI)..... 14

2.3.4 Waveform 1.4: Cascade Mode..... 15

2.3.5 Waveform 1.5: Priority Rotation..... 15

2.4 Navigation..... 17

3 pic_8259 – Register Map..... 18

3.1 Overview..... 18

3.2 Functional Description..... 19

3.2.1 Register Map..... 19

3.2.2 Global Configuration..... 20

3.2.3 Initialization Command Words (ICW)....21
3.2.4 Operation Command Words (OCW)....23
3.2.5 Status / Readback Registers....26
3.3 Design Notes....29
3.4 Navigation....30

# List of Figures

Figure 1.1: PIC 8259 Block Diagram.....10

# List of Tables

*No tables in this document.*

# List of Waveforms

Waveform 1.1: Interrupt Request....13
Waveform 1.2: Interrupt Acknowledge Sequence....13
Waveform 1.3: End-of-Interrupt (EOI)....14
Waveform 1.4: Cascade Mode....15
Waveform 1.5: Priority Rotation....15

# 1 Pic 8259 Mas Index

**Generated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 2 pic_8259 – Overview

## 2.1 Overview

The pic_8259 is an 8259A-compatible Programmable Interrupt Controller with an APB interface. It handles interrupt management for legacy PC-compatible systems: eight request lines in, one interrupt line out, with masking, priority resolution, and an initialization sequence in between.

Implemented in the current RTL:

- 8 interrupt inputs (`irq_in[7:0]`) with a single `int_out` line. `irq_in` may be asynchronous – it passes a `SYNC_STAGES`-flop synchronizer (default 2) before the edge detector, so a pin transition costs `SYNC_STAGES + 1` clocks before `int_out` reflects it
- Fully-decoded 32-bit APB register file (no legacy A0 two-port model). Only 0x00-0x2C are decoded; everything else in the 4 KB window is dropped with `PSLVERR`
- Acknowledge by read: a read of PIC_INTA (0x2C) stands in for the INTA bus cycle. It returns `{vector_base[7:3], irq}` with a valid bit, sets the ISR bit, and clears an edge-mode IRR bit
- Live IRR and ISR with the fully nested rule: an in-service level blocks itself and every lower-priority level until its EOI; a higher-priority level preempts
- All eight OCW2 commands: non-specific, specific and rotating EOI, set-priority, rotate-on-AEOI arm/disarm, and automatic EOI (ICW4 AEOI)
- Special mask mode (OCW3)
- Edge or level triggering
- Interrupt masking (IMR / OCW1)

Now the other half of the story. The following register bits are software-visible storage with no hardware effect – they are kept so a classic ICW/OCW sequence runs unchanged, and that is all they do (see the register map design notes):

- Special fully nested mode (ICW4 SFNM)
- Buffered mode (ICW4 BUF)
- OCW3 read-register-select and poll. IRR and ISR have dedicated read-only registers, and PIC_INTA answers the poll question with the acknowledge folded in
- `auto_reset_init` (PIC_CONFIG bit 2) affects only whether `init_mode` clears itself after ICW4

This is the block after the 2026-09-09 fixes (issue #50); before them there was no acknowledge path at all.

### 2.1.1 Parameters

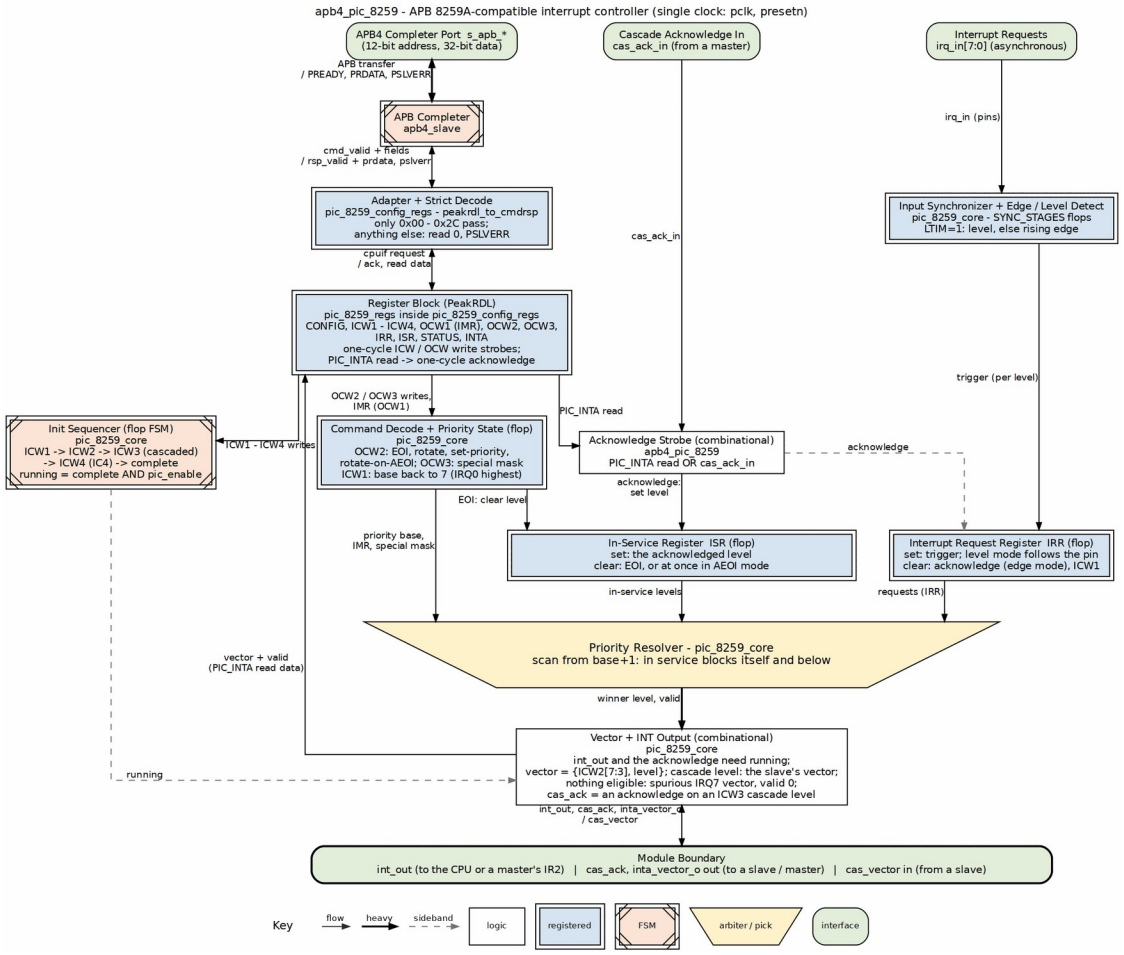
| Parameter | Type | Default | Description |
|---|---|---|---|
| SYNC_STAGES | int | 2 | `irq_in` synchronizer depth in flops. Minimum 2 (a simulation-time guard rejects less) |

### 2.1.2 Applications

- PC-compatible interrupt management
- Legacy device support
- x86 system integration

## 2.2 Functional Description

### 2.2.1 Figure 1.1: PIC 8259 Block Diagram




*PIC 8259 Block Diagram*

### 2.2.2 Register Summary

The block uses a fully-decoded 32-bit register file, not the legacy A0 two-port model. See Chapter 5: Register Map for full field definitions.

| Offset | Register | Access | Description |
|---|---|---|---|
| 0x00 | PIC_CONFIG | RW | Global configuration (pic_enable, init_mode, auto_reset_init) |
| 0x04 | PIC_ICW1 | WO | Initialization Command Word 1 |

| Offset | Register | Access | Description |
|---|---|---|---|
| 0x08 | PIC_ICW2 | WO | Initialization Command Word 2 (vector base) |
| 0x0C | PIC_ICW3 | WO | Initialization Command Word 3 (cascade config) |
| 0x10 | PIC_ICW4 | WO | Initialization Command Word 4 |
| 0x14 | PIC_OCW1 | RW | Interrupt Mask Register (IMR) |
| 0x18 | PIC_OCW2 | WO | EOI / priority command |
| 0x1C | PIC_OCW3 | WO | Special mask (read-select and poll are storage only) |
| 0x20 | PIC_IRR | RO | Interrupt Request Register |
| 0x24 | PIC_ISR | RO | In-Service Register |
| 0x28 | PIC_STATUS | RO | Initialization state / diagnostics |
| 0x2C | PIC_INTA | RO | Interrupt acknowledge by read (side-effecting) |

The PIC is disabled at reset - firmware must BOTH complete the ICW initialization sequence (ICW1 -> ICW2 -> ICW3 if cascaded -> ICW4 if requested; PIC_STATUS.init_complete=1) AND set `pic_enable` (PIC_CONFIG bit 0) before any interrupt can be requested or delivered – out of reset the init FSM sits in INIT_IDLE and IRR never updates. OCW2 and OCW3 commands are ignored until both conditions hold.

### 2.2.3 Interrupt Priority

| IRQ | Default Priority |
|---|---|
| IR0 | Highest (0) |
| IR1 | 1 |
| IR2 | 2 (the conventional cascade input on a master) |
| IR3 | 3 |
| IR4 | 4 |
| IR5 | 5 |

| IRQ | Default Priority |
|---|---|
| IR6 | 6 |
| IR7 | Lowest (7) |

### 2.2.4 Priority Modes

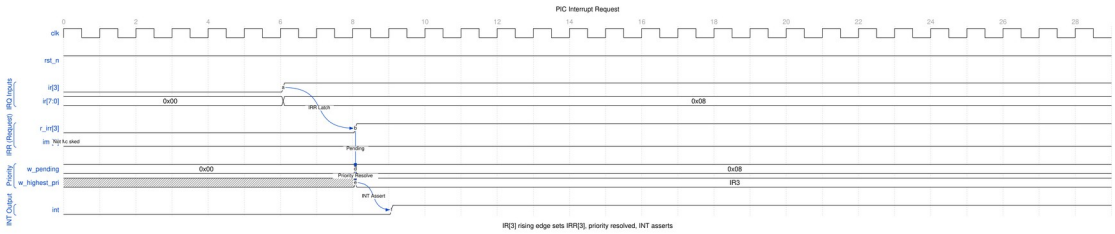
- **Fixed Priority**: IR0 highest, IR7 lowest (the reset and post-ICW1 order)
- **Rotating Priority**: the level just retired becomes lowest – rotate on non-specific EOI (0xA0), rotate on specific EOI (0xE0-0xE7), or once per acknowledge with rotate-on-AEOI armed (0x80) in AEOI mode
- **Specific Priority**: set-priority (0xC0-0xC7) names the lowest level

In every mode an in-service level blocks itself and all lower-priority levels until its EOI, and `int_out` asserts only for an unmasked request that outranks everything in service. Special mask mode (OCW3) relaxes that for an in-service level that software has also masked.

## 2.3 Waveforms

### 2.3.1 Waveform 1.1: Interrupt Request

Shows an IRQ input assertion triggering the interrupt process.

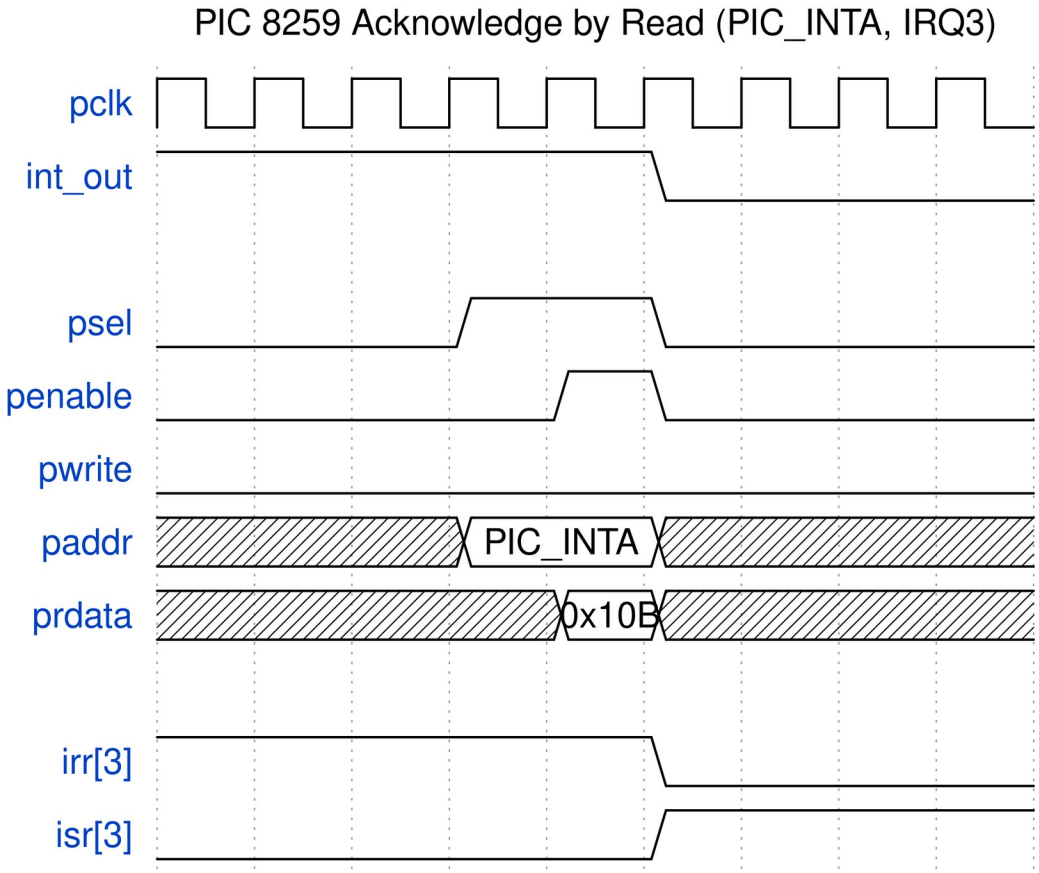



*PIC Interrupt Request*

When an IR pin asserts, the synchronized input sets the corresponding IRR bit (on the rising edge in edge mode; for as long as the pin is high in level mode). The priority resolver selects the highest priority unmasked request that outranks every in-service level and asserts INT to the CPU. A masked request stays in the IRR and never reaches INT until it is unmasked.

### 2.3.2 Waveform 1.2: Interrupt Acknowledge Sequence

The acknowledge, as an APB read of PIC_INTA.

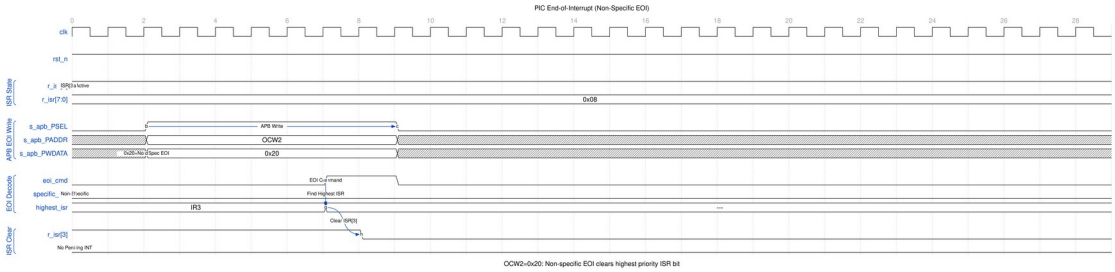



*PIC Interrupt Acknowledge*

This block has no INTA pin; a single APB read of PIC_INTA (0x2C) does the work of both classic INTA pulses. In the access cycle the read data carries the vector (base + IR number) with bit 8 set to say a request was acknowledged, and at the end of that same cycle the core transfers the level from IRR to ISR (in edge mode the IRR bit clears; in level mode it keeps following the pin). The data returned is the pre-acknowledge state. INT drops in the next cycle unless a higher-priority request is still waiting. With nothing pending the read returns valid=0 and the spurious IRQ7 vector, and changes nothing.

### 2.3.3 Waveform 1.3: End-of-Interrupt (EOI)

Software clears the in-service bit with an EOI command.

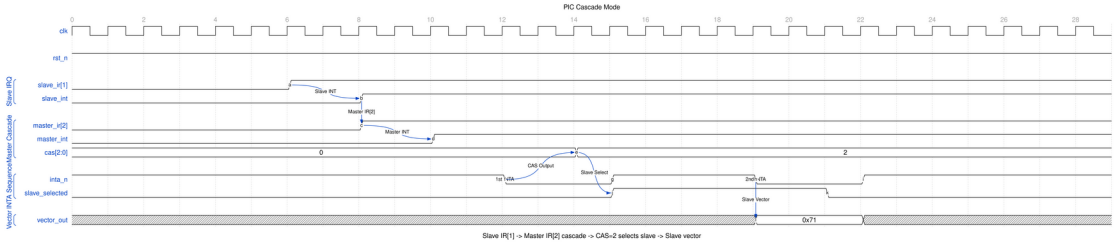



## *PIC EOI*

Non-specific EOI (0x20) clears the highest priority ISR bit – under the fully nested rule, the level acknowledged most recently; under special mask mode, the highest in-service level that is not masked. Specific EOI (0x60-0x67) clears a designated IR. Either is a no-op when nothing is in service. Once the bit clears, the levels it was blocking become eligible again, so INT reasserts if a lower-priority request was waiting; in AEOI mode the acknowledge itself retires the level and no EOI is written.

### 2.3.4 Waveform 1.4: Cascade Mode

Master-slave configuration for 15 IRQ sources.



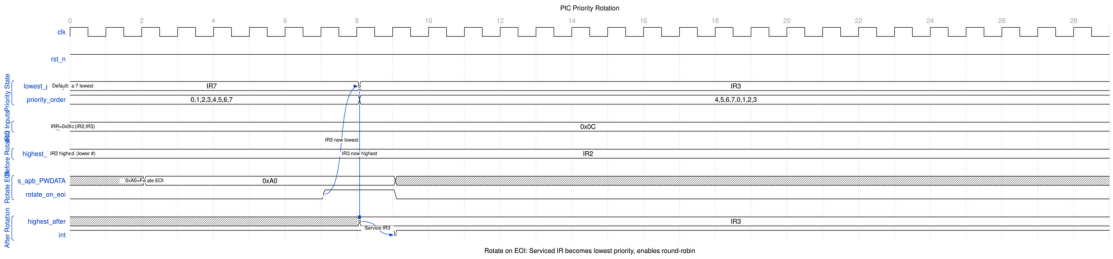

## *PIC Cascade*

Slave INT connects to master IR2. During INTA, master outputs cascade select (CAS) lines. Slave with matching ID provides the interrupt vector.

> Note: cascade IS implemented (RLB/pic_8259 TASK-001), but this block has no CAS or SP/EN pins, so the `cascade_sel` step above is architectural rather than literal. Acknowledge here is an APB read of PIC_INTA, not an INTA pulse pair, so there is no second pulse on which to broadcast a slave ID. The equivalent, and what the RTL does: when the acknowledged level is one ICW3 marks as cascaded, the master returns the SLAVE's vector and forwards the acknowledge down. Covered by `dv/tests/test_pic_8259_cascade.py` (RLB/pic_8259 TASK-001).

### 2.3.5 Waveform 1.5: Priority Rotation

Automatic priority rotation for equal-service scheduling.




## *PIC Priority Rotation*

Rotate-on-EOI (0xA0) retires the highest-priority in-service level and makes it the lowest priority, implementing round-robin scheduling among interrupt sources. Rotate on specific EOI (0xE0-0xE7) does the same for a named level, set-priority (0xC0-0xC7) moves the base without an EOI, and with rotate-on-AEOI armed (0x80) each acknowledge in AEOI mode rotates exactly once. ICW1 restores the fixed IR0-highest order.

## 2.4 Navigation

**Next:** 02_architecture.md *(planned, not yet written)*

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 3 pic_8259 – Register Map

## 3.1 Overview

Forget the legacy two-port, A0-based interface of the original 8259A – this block doesn't have one. What you get is a fully-decoded, 32-bit-aligned APB register file: each ICW/OCW and every status register sits at its own dedicated offset, with no A0 pin and no OCW3 read-select multiplexing. The one thing APB cannot give you is an INTA bus cycle, so the acknowledge is a READ of PIC_INTA (0x2C) – the same trick the ARM PL190 VIC plays with `VICVectAddr`. Reading that register has side effects; its section below spells them out.

Decode is strict. Only the twelve offsets 0x000 through 0x02C exist. Every other address in the 4 KB window is dropped – a write is ignored, a read returns 0x0000_0000 – and the transfer completes with `PSLVERR` asserted. There is no aliasing: 0x044 is not ICW1 and 0x054 is not the IMR, they are errors. (Earlier RTL decoded six address bits for storage while the side-effect strobes compared all twelve, so an aliased write landed in storage and fired nothing; fixed 2026-09-09, issue #50.)

## 3.2 Functional Description

### 3.2.1 Register Map

| Offset | Register | Access | Reset | Description |
|---|---|---|---|---|
| 0x00 | PIC_CONFIG | RW | 0x0000_0004 | Global configuration and control (gates all operation) |
| 0x04 | PIC_ICW1 | WO | — | Initialization Command Word 1 (edge/level, single, ICW4-needed) |
| 0x08 | PIC_ICW2 | WO | — | Initialization Command Word 2 (interrupt vector base) |
| 0x0C | PIC_ICW3 | WO | — | Initialization Command Word 3 (cascade config) |
| 0x10 | PIC_ICW4 | WO | — | Initialization Command Word 4 (mode bits) |
| 0x14 | PIC_OCW1 | RW | 0x0000_00FF | Operation Command Word 1 - Interrupt Mask Register (IMR) |
| 0x18 | PIC_OCW2 | WO | — | Operation Command Word 2 (EOI / priority command) |
| 0x1C | PIC_OCW3 | WO | — | Operation Command Word 3 (special mask; read-select and poll are storage only) |
| 0x20 | PIC_IRR | RO | 0x0000_0000 | Interrupt Request Register |
| 0x24 | PIC_ISR | RO | 0x0000_00 | In-Service Register |

| Offset | Register | Access | Reset | Description |
|---|---|---|---|---|
| | | | 00 | |
| 0x28 | PIC_STATUS | RO | — | Initialization state and diagnostics |
| 0x2C | PIC_INTA | RO | — | Interrupt acknowledge by read – reading it acknowledges the pending interrupt |

Anything else in the window: dropped with PSLVERR.

**Access notes:** PIC_ICW1-ICW4, PIC_OCW2, and PIC_OCW3 are write-only in the RTL - their read-back paths are tied to zero, so reading these offsets returns 0x0000_0000. PIC_CONFIG, PIC_OCW1 (IMR), PIC_IRR, PIC_ISR, PIC_STATUS and PIC_INTA return meaningful data on a read. PIC_INTA and the two status registers are wires, not storage – what you read is the core's state in that cycle.

### 3.2.2 Global Configuration

#### *3.2.2.1 PIC_CONFIG (Offset 0x00, RW)*

The PIC is disabled out of reset. Firmware **must** complete the ICW initialization sequence (through PIC_STATUS.init_complete=1) AND set `pic_enable` before any interrupt request can propagate - IRR only updates once the init FSM reaches INIT_COMPLETE. `pic_enable` gates delivery, not observation: while it is 0 the core forces `int_out` low and refuses to acknowledge, but IRR keeps capturing requests and ISR keeps whatever was in service, so a disable/enable cycle mid-handler neither loses a request nor silently retires a level. The edge detector tracks the synchronized inputs at all times, so a request that drops and rises again inside a disabled window is seen, and enabling never manufactures an edge from a line that was already high. A line that rises between ICW1 and ICW4 is not latched (IRR capture waits for INIT_COMPLETE, and ICW1 clears IRR anyway) and is not manufactured either. OCW2 and OCW3 commands are gated the same way: a write issued before initialization completes, or with `pic_enable=0`, changes the register storage and nothing else – it cannot rotate the priority base or set special mask mode ahead of time.

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 0 | pic_enable | RW | 0 | Master enable (0=disabled, 1=enabled) |
| 1 | init_mode | RW | 0 | Initialization request. Taken on its 0 -> 1 edge; consumed by the next ICW1 write |

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 2 | auto_reset_init | RW | 1 | Hardware clears init_mode when ICW4 is written |
| 31:3 | Reserved | RO | 0 | Reserved |

`init_mode` is a start REQUEST, not a mode level. The core takes it on its rising edge and arms a re-initialization; the next ICW1 write consumes the request and begins the sequence. If the PIC was already initialized, arming drops the init FSM from INIT_COMPLETE back to INIT_IDLE (so PIC_STATUS.init_complete reads 0 until the new sequence finishes), which is the bit's practical use – letting software watch a re-initialization from step 0. Initialization does not otherwise need it: an ICW1 write from INIT_IDLE or INIT_COMPLETE starts the sequence on its own. Holding the bit at 1 afterwards changes nothing, so `auto_reset_init=0` is a legal configuration and INIT_COMPLETE holds with `init_mode` still set.

With `auto_reset_init=1` the hardware clears `init_mode` in the cycle the ICW4 write takes effect. The clear is a hardware-precedence `hwclr` in the register block, so a PIC_CONFIG write landing in that same cycle cannot swallow it: the software write still takes for `pic_enable` and `auto_reset_init`, and the clear still wins for bit 1. `auto_reset_init` has no other effect.

Re-initialization: an ICW1 write clears IRR, ISR and special mask mode, disarms rotate-on-AEOI and restores the priority base to 7 (IRQ0 highest), as it does on a real 8259A. Two departures from the datasheet remain. First, only INIT_IDLE and INIT_COMPLETE honor ICW1 as a sequence start – an ICW1 write while the FSM is mid-sequence (WAIT_ICW2/3/4) clears the state above but does not restart the step counter. Second, ICW1 does not touch the ICW4 storage, so an IC4=0 re-initialization keeps whatever ICW4 last held (reset value uPM=1, AEOI=0) where a real part would assume zeros; write ICW4 explicitly if AEOI must be off.

### 3.2.3 Initialization Command Words (ICW)

All ICW registers are write-only; reading them returns 0.

#### *3.2.3.1 PIC_ICW1 (Offset 0x04, WO)*

| Bit | Name | Reset | Description |
|---|---|---|---|
| 0 | IC4 | 0 | 1 = ICW4 needed, 0 = ICW4 not needed |
| 1 | SNGL | 0 | 1 = single mode (no ICW3 in the sequence), 0 = cascade mode (ICW3 expected) |
| 2 | ADI | 0 | Call address interval (8080/8085 mode only; storage) |
| 3 | LTIM | 0 | 1 = level triggered, 0 = edge |

| Bit | Name | Reset | Description |
|---|---|---|---|
| | | | triggered |
| 4 | ICW1 Marker | 1 | Resets to 1 (8259A ICW1 identifier); plain writable storage – hardware does not enforce or check it |
| 31:5 | Reserved | 0 | Reserved (the RTL implements no bits above bit 4) |

Note: the RTL stores only bits [4:0]. The legacy A7-A5 vector bits of an MCS-80 8259A are not implemented here. SNGL's only hardware effect is whether the init FSM waits for ICW3.

#### *3.2.3.2 PIC_ICW2 (Offset 0x08, WO)*

| Bits | Name | Reset | Description |
|---|---|---|---|
| 7:0 | vector_base | 0x00 | Interrupt vector base. The vector returned by PIC_INTA is `{vector_base[7:3], irq[2:0]}`; bits [2:0] written here never reach it |
| 31:8 | Reserved | 0 | Reserved |

#### *3.2.3.3 PIC_ICW3 (Offset 0x0C, WO)*

Cascade configuration, and it is READ by the core (RLB/pic_8259 TASK-001). Write-only to software, which is the real 8259A behaviour – a read returns 0.

This block has no CAS or SP/EN pins: it acknowledges by an APB read of PIC_INTA, so there is no second INTA pulse on which a master could broadcast a slave ID. The equivalent is that a master whose acknowledged level is marked here returns the SLAVE's vector and forwards the acknowledge down. Covered by `dv/tests/test_pic_8259_cascade.py` (RLB/pic_8259 TASK-001).

**Master mode:**

| Bits | Description |
|---|---|
| 7:0 | Bitmap of IR lines that have a slave attached (1 = slave present) |

**Slave mode:**

| Bits | Description |
|---|---|
| 2:0 | Slave ID (cascade input number |

| Bits | Description |
|---|---|
| | 0-7) |

#### *3.2.3.4 PIC_ICW4 (Offset 0x10, WO)*

| Bit | Name | Reset | Description |
|---|---|---|---|
| 0 | uPM | 1 | 1 = 8086/8088 mode, 0 = 8080/8085 mode (storage; the vector is always the 8086 form) |
| 1 | AEOI | 0 | Automatic EOI: the acknowledge retires the level at once (see PIC_INTA) |
| 3:2 | BUF (M/S) | 00 | Buffered-mode select: 00=non-buffered, 10=buffered slave, 11=buffered master. Storage only |
| 4 | SFNM | 0 | Special fully nested mode. Storage only – the ordinary fully nested rule runs regardless |
| 31:5 | Reserved | 0 | Reserved |

### 3.2.4 Operation Command Words (OCW)

#### *3.2.4.1 PIC_OCW1 - Interrupt Mask Register (Offset 0x14, RW)*

| Bits | Name | Access | Reset | Description |
|---|---|---|---|---|
| 7:0 | imr | RW | 0xFF | Interrupt mask for IRQ0-7: 1 = masked (disabled), 0 = unmasked (enabled) |
| 31:8 | Reserved | RO | 0 | Reserved |

Reset masks all eight interrupts.

The register field is the ONE copy of the mask – the core reads it directly, there is no mirror – so readback is exact from the cycle after the write and the new mask reaches the priority resolver in the same cycle. A masked request stays in the IRR (edge mode) or keeps following its pin (level mode) and is delivered once unmasked; `int_out` never asserts for a masked level. The IMR is not written by the init sequence; program it after ICW4.

#### *3.2.4.2 PIC_OCW2 (Offset 0x18, WO)*

| Bits | Name | Reset | Description |
|---|---|---|---|
| 2:0 | irq_level (L2-L0) | 0 | IRQ level for specific EOI or set-priority |
| 4:3 | Reserved | 0 | Reserved (OCW2 identifier field in a legacy 8259A) |
| 7:5 | eoi_cmd (R,SL,EOI) | 0 | EOI / rotation command (see table) |
| 31:8 | Reserved | 0 | Reserved |

**EOI / rotation command encodings** (bits [7:5] = R, SL, EOI), as executed by the RTL. Every command takes effect on the write and only while the PIC is initialized and enabled:

| R | SL | EOI | Command | Effect |
|---|---|---|---|---|
| 0 | 0 | 0 | Rotate on auto EOI (clear) | Disarms rotate-on-AEOI |
| 0 | 0 | 1 | Non-specific EOI | Clears the highest-priority in-service bit (in the current rotated order); under special mask mode, the highest-priority in-service bit among the levels that are not masked. No-op when nothing qualifies |
| 0 | 1 | 1 | Specific EOI | Clears ISR[L2-L0] |
| 0 | 1 | 0 | No operation | Same as a real 8259A |
| 1 | 0 | 0 | Rotate on auto EOI (set) | Arms rotate-on-AEOI: in AEOI mode each acknowledge then makes the acknowledged level lowest priority |
| 1 | 0 | 1 | Rotate on non-specific | Clears the highest- |

| R | SL | EOI | Command | Effect |
|---|---|---|---|---|
| | | | EOI | priority in-service bit AND makes that level lowest priority. No-op when ISR is empty – the base does not move |
| 1 | 1 | 0 | Set priority | L2-L0 becomes lowest priority; ISR untouched |
| 1 | 1 | 1 | Rotate on specific EOI | Clears ISR[L2-L0] and makes L2-L0 lowest priority |

“Lowest priority” is the priority base: with base = b the order, highest first, is b+1, b+2, … , 7, 0, … , b (mod 8). The reset and post-ICW1 base is 7, giving the familiar IRQ0-highest order. This is a rotation of the level index, not a reflection – with base = 3 the order is 4,5,6,7,0,1,2,3, so IRQ4 outranks IRQ2.

Under the fully nested rule the highest-priority in-service level is the one acknowledged most recently, so a non-specific EOI from a nested handler retires the right level without the handler knowing which one it is. Special mask mode is the one place that ordering breaks – a lower-priority level can be acknowledged on top of a masked in-service one – so the non-specific EOI scan skips in-service levels whose IMR bit is set, exactly the levels SMM has stopped blocking. A handler that masked itself and set SMM is not retired by the EOI of the interrupt it let in.

A write with no byte enables asserted (PSTRB = 0) changes nothing and executes nothing: the command strobes are qualified by the byte enables, so such a write cannot replay the command already stored in OCW2 or ICW1.

#### *3.2.4.3 PIC_OCW3 (Offset 0x1C, WO)*

| Bits | Name | Reset | Description |
|---|---|---|---|
| 1:0 | read_reg_cmd (RIS,RR) | 00 | Read-register select: 00=no action, 10=read IRR, 11=read ISR. Storage only – IRR and ISR have dedicated registers |
| 2 | P | 0 | Poll command. Storage only – PIC_INTA supersedes it |
| 4:3 | OCW3 Marker | 01 | Resets to 01 (OCW3 identifier); plain writable storage – not |

| Bits | Name | Reset | Description |
|---|---|---|---|
| | | | enforced or checked |
| 6:5 | ESMM,SMM | 00 | Special mask mode: 10=reset special mask, 11=set special mask; 0x = no change |
| 31:7 | Reserved | 0 | Reserved |

Special mask mode changes what an in-service level blocks. Normally a level in service blocks itself and every lower-priority level until its EOI. With SMM set, an in-service level whose IMR bit is also set stops blocking – requests from every other unmasked level, lower as well as higher, are delivered while it is in service. An in-service level that is NOT masked still blocks as usual. So the idiom is the datasheet one: a handler that wants to let other interrupts in while it runs masks its own level and sets SMM; clearing SMM (or unmasking the level) restores the fully nested rule. SMM takes effect on the write, while the PIC is initialized and enabled, and ICW1 clears it.

### 3.2.5 Status / Readback Registers

#### *3.2.5.1 PIC_IRR - Interrupt Request Register (Offset 0x20, RO)*

| Bits | Name | Reset | Description |
|---|---|---|---|
| 7:0 | irr | 0 | Pending interrupt requests for IRQ0-7 (1 = request pending) |
| 31:8 | Reserved | 0 | Reserved |

IRR reflects requests before masking. In level mode each bit follows its (synchronized) IRQ pin, and the acknowledge leaves it alone – a pin held high re-requests as soon as the level is retired, which is what level-triggered means. In edge mode a bit sets on the rising edge of the synchronized input and clears when that level is acknowledged through PIC_INTA; a fresh edge landing in the acknowledge cycle is kept, not swallowed. An EOI never touches the IRR. Both modes clear the whole register on an ICW1 write or when `pic_enable` drops.

#### *3.2.5.2 PIC_ISR - In-Service Register (Offset 0x24, RO)*

| Bits | Name | Reset | Description |
|---|---|---|---|
| 7:0 | isr | 0 | In-service bits for IRQ0-7 (1 = interrupt being serviced) |
| 31:8 | Reserved | 0 | Reserved |

The live in-service register. A bit sets when its level is acknowledged by a PIC_INTA read and clears on the matching EOI (non-specific, specific, or either rotate variant), on an ICW1 write, or when `pic_enable` drops. In AEOI mode the acknowledge sets and clears the bit in the same cycle, so it is never observable. Several bits may be set at once when handlers nest.

#### *3.2.5.3 PIC_STATUS (Offset 0x28, RO)*

Initialization-state and diagnostic readback. This register is the only way to observe the init sequence progress from software.

| Bits | Name | Reset | Description |
|---|---|---|---|
| 0 | init_complete | - | 1 = initialization complete, 0 = in init sequence |
| 3:1 | icw_step | - | Current ICW step (0 = not initialized, 4 = complete) |
| 4 | int_output | - | Current state of the INT output pin |
| 7:5 | highest_priority | - | Highest-priority pending unmasked request in the current rotated order, ignoring the ISR – “which level is next”, not “which level may interrupt now”. Reads 0 when nothing is pending |
| 31:8 | Reserved | 0 | Reserved |

#### *3.2.5.4 PIC_INTA - Interrupt Acknowledge (Offset 0x2C, RO, side-effecting)*

The 8259A moves a request from the IRR to the ISR during the CPU’s INTA bus cycle and puts the vector on the data bus in the second pulse. APB has neither pulse, so a read of PIC_INTA does both jobs in one transfer. Writes are ignored (no side effect, no error).

| Bits | Name | Reset | Description |
|---|---|---|---|
| 7:0 | vector | - | `{ICW2.vector_base[7:3], irq[2:0]}` of the acknowledged level when valid=1; `{vector_base[7:3], 3'b111}` (spurious IRQ7) when valid=0 |
| 8 | valid | - | 1 = a request was acknowledged and vector names it; 0 = nothing was pending, the read had no side effects |

| Bits | Name | Reset | Description |
|---|---|---|---|
| 31:9 | Reserved | 0 | Reserved |

If an unmasked, unblocked request is pending – the same condition that drives `int_out`, so a read can never acknowledge something the pin was not offering – the read:

- returns `valid = 1` and that level's vector. The data is the state BEFORE the acknowledge: PIC_INTA is a wire in the register block and the bridge captures it in the same cycle the core registers the side effect;
- sets `ISR[irq]`, which from the next cycle blocks that level and every lower-priority one until EOI;
- clears `IRR[irq]` in edge mode. In level mode the IRR follows the pin and is left alone;
- in AEOI mode also clears `ISR[irq]` in the same cycle, and if rotate-on-AEOI is armed (OCW2 0x80) moves the priority base to the acknowledged level – exactly once per acknowledge.

If nothing is pending the read returns `valid = 0` with the spurious vector and changes nothing. That is the 8259A's spurious-IRQ7 convention; check `valid` rather than decoding the vector.

`int_out` drops in the cycle after the acknowledge unless another request of higher priority than everything now in service is waiting – with the in-service block, that means a HIGHER-priority level. A second edge on the level just acknowledged sets its IRR bit again but is held off until its EOI.

**Programming sequence.** With `pic_enable=1` and initialization complete:

1. `int_out` asserts (PIC_STATUS.int_output reads 1).
2. Read PIC_INTA. If `valid=0`, the interrupt was spurious – return without an EOI. Otherwise `vector[2:0]` is the level and `vector` is the full interrupt number.
3. Service the device. A higher-priority request can assert `int_out` during this step; a handler that re-enables CPU interrupts may re-enter at step 2 for it, and that nested handler's EOI retires the nested level first.
4. Write PIC_OCW2: non-specific EOI (0x20) or specific EOI (0x60 | level), or the rotate variants (0xA0, 0xE0 | level) for round-robin service. In AEOI mode skip this step – the acknowledge already retired the level.

## 3.3 Design Notes

The register file above matches the RTL exactly. The behaviors that follow are the ones a datasheet reader would otherwise assume or overlook:

- **Priority and the INT pin.** A level is eligible when it is requesting, unmasked, and no level of equal or higher priority is in service. `int_out` is the OR of eligible levels (while initialized and enabled); the same predicate chooses what PIC_INTA acknowledges. An in-service level blocks itself and everything below it until its EOI; a higher-priority level still preempts. Special mask mode is the one exception, described under OCW3.
- **Timing.** `irq_in` may be asynchronous. It passes a `SYNC_STAGES`-flop synchronizer (default 2, minimum 2) before the edge detector, so a pin transition reaches the IRR `SYNC_STAGES + 1` clocks later and `int_out`, which is combinational from the IRR, IMR and ISR, follows in that cycle – about three `pclk` periods at the default. Register writes (IMR, OCW2, OCW3) act one cycle after the APB access phase; PIC_INTA acts in the access cycle.
- **Software-visible storage with no hardware effect.** ICW4 BUF and SFNM, OCW3 read-register-select and poll, and ICW1 ADI are stored and never read by the core. There is no special fully nested mode. ICW3 is NO LONGER in this list (RLB/pic_8259 TASK-001): it is read by the core and selects the cascade level. There are still no CAS or SP/EN pins – see the PIC_ICW3 notes above for the acknowledge-by-read equivalent. IRR and ISR are dedicated registers, so the read-select command has nothing to select; the poll command is superseded by PIC_INTA, which returns the same “is there an interrupt, and which” answer with the acknowledge folded in. `auto_reset_init` affects only the clearing of `init_mode`.
- **History.** The acknowledge-by-read path, live ISR, EOI/nesting/SMM semantics, one-shot rotate-on-AEOI, single-copy IMR, gated OCW2/OCW3, edge- taken `init_mode`, strict decode with `PSLVERR` and the input synchronizer all landed together on 2026-09-09 (issue #50). Before that the block had no acknowledge at all, so the ISR never set and an edge request never cleared.

## 3.4 Navigation

**Back to:** PIC 8259 Specification Index